Man-in-the-Disk (MitD) Vulnerability

A Man-in-the-Disk (MitD) vulnerability arises when an application stores sensitive data or temporary files on external or shared storage. This exposes the data to other applications or attackers who can read, modify, or replace it before the original app can securely process it.

Shared Storage Exposure

Sensitive files are placed on publicly accessible file systems (e.g., SD cards, /tmp).

Attacker Intervention

Malicious actors intercept these files, reading, altering, or replacing them.

Compromised Application

The legitimate application then trusts and processes the tampered data.

World-Writable /tmp File Injection

A classic Man-in-the-Disk attack vector: when an application writes to `/tmp` without secure file creation, an attacker can pre-position a symlink and silently redirect writes to critical system files.

Vulnerable Code Pattern

App writes to `/tmp/app.tmp` without using `mkstemp()` or enforcing strict permissions — assuming the path is safe.

The Attack Sequence

1. **Attacker runs: `ln -s /etc/passwd /tmp/app.tmp`**
2. **Legitimate program writes to `/tmp/app.tmp`**
3. **Kernel follows the symlink — `/etc/passwd` is overwritten**

Root Cause

`/tmp` is shared and world-writable — any user can create or replace files there before your app does.

Impact

Privilege escalation and full system compromise via overwritten system files.

Fix

Always use `mkstemp()`, `O_EXCL`, and restrict permissions. Never assume `/tmp` is safe.

Temporary File Modification in Desktop Apps

When a desktop application stages output through a shared or world-accessible temporary path, any concurrent process — or another logged-in user — can intervene before the file is finalised.

Scenario

A desktop app exports a document via:

`C:\Users\Public\report.tmp`

The file sits in a shared, accessible location while the export is in progress.

What an Attacker Injects

Malicious Macros

Execute arbitrary code when the document is opened by a privileged user.

Altered Financial Data

Silent manipulation of figures — no visible indicators of tampering.

2

Attack Surfaces

Macro injection and data manipulation — both achievable via a single file swap.

0

User Interaction Required

The attacker operates silently in the background — no prompts, no alerts.

Build Pipeline & CI Artifact Tampering

Shared build directories are a high-value target. If an attacker gains even limited access to a build environment, they can swap artifacts between compilation and signing — injecting backdoors that propagate silently downstream.

The pipeline has no reason to distrust the artifact — it was placed exactly where the system expects it. Signature checks only protect integrity *after* the swap, not before.

This is Man-in-the-Disk applied to DevOps: the shared filesystem becomes the interception layer between build and release. Enforce artifact checksums and use immutable build environments to close this gap.

Log File Poisoning → Code Execution

Logs are often treated as inert output — but when a second component parses them as structured input, they become an injection surface.

The Attack Chain

1. App writes user-controlled content to `/var/log/app.log`
2. Attacker crafts input that embeds commands or structured payloads
3. A downstream parser interprets log entries as executable instructions

Impact

- Command injection into downstream processes
- Logic manipulation via poisoned structured data

Installer & Updater Using Shared Directory

Poorly designed updaters download executables to predictable, user-accessible paths — and then trust whatever is there at execution time. This is one of the most commonly exploited MitD patterns in the wild.

Vulnerable Pattern

Updater downloads to `C:\Temp\update.exe`, then executes it — with no integrity check between download and launch.

What Goes Wrong

Attacker Monitors

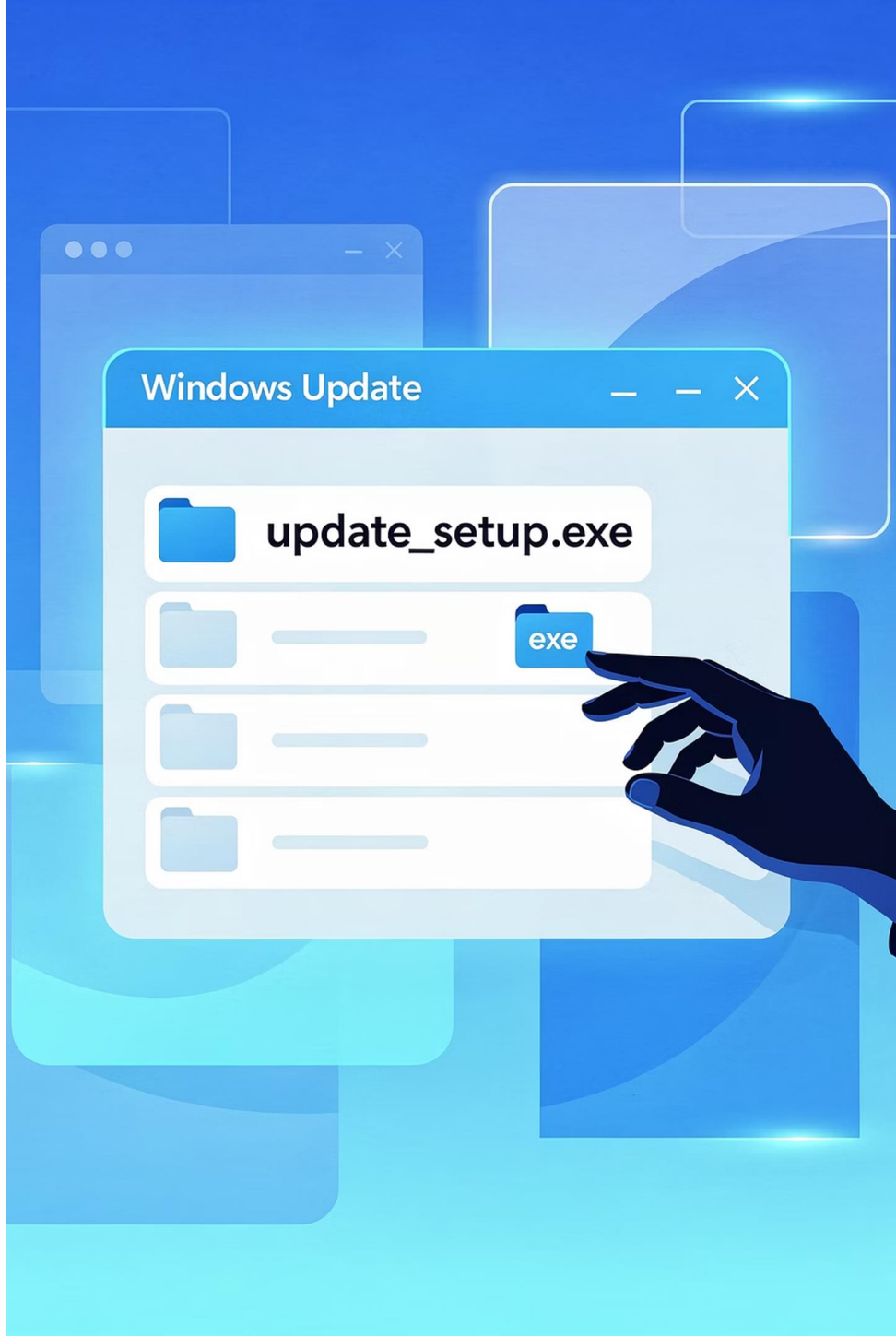
Watches for the download to complete at the known path.

Attacker Replaces

Swaps `update.exe` with a malicious binary before the launcher runs.

System Executes

Malware runs with user or admin privileges — fully trusted by the OS.



Shared Network Drive Tampering

Applications that read configuration from shared network drives (SMB/NFS) are vulnerable whenever an attacker can write to — or position themselves between — the share and the application.

Attack Vector

App reads `\\shared-drive\configs\settings.json`. Attacker modifies the file between application reads — no network interception required, just write access to the share.

What Can Be Manipulated

- Feature flags and access controls
- Credential endpoints — redirect auth to attacker-controlled servers
- Role or permission assignments

Database File on Shared Storage

Storing a SQLite database on a shared filesystem moves the authentication boundary from the application layer to the filesystem layer — wherever the file is accessible, the entire database is attackable.

Scenario

SQLite DB lives at `/mnt/shared/app.db` — accessible to any process or user with filesystem access, bypassing all application-level auth.

Direct Manipulation Payloads

Add Admin User

Insert a row directly into the users table — no password required.

Alter Permissions

Escalate any existing user's role without touching the application.

Corrupt Data

Overwrite or delete records to cause application failure or cover tracks.

Plugin & DLL Search Order Hijacking

When Windows resolves a DLL or plugin by name, it searches directories in a defined order — starting with the application's own directory. If that directory is writable, it becomes the disk-layer "man in the middle" for dependency loading.

The Vulnerability

App calls `LoadLibrary("plugin.dll")`. Windows searches writable directories first. Attacker drops a malicious `plugin.dll` there — it loads instead of the legitimate one.

No network access, no credential theft. Just a file drop.

The disk becomes the man in the middle of dependency loading. Audit all writable directories in your application's DLL search path and enforce directory ACLs rigorously.

Unifying Pattern: MitD Beyond Mobile

All the attack scenarios covered share the same fundamental flaw — trusting mutable, shared storage without verification.

❌ Trusting Mutable Storage

- **Shared directories: /tmp, C:\Temp**
- **Network shares (SMB/NFS)**
- **External drives & SD cards**
- **Build artifact directories**

❌ Missing Protections

- **No integrity checks**
- **No secure file permissions**
- **No atomic operations**
- **No file locking**

Use secure temp file APIs

`mkstemp()`, `O_CREAT | O_EXCL`

Restrict permissions

`chmod 0600`, **strict ACLs**

Avoid shared directories for sensitive data

Minimize exposure where possible

Validate integrity

Hash or signature checks before use

Use atomic writes + file locking

Ensure operations complete without interruption

Never trust intermediate files

Always re-verify data at each critical step

Any time storage is shared and mutable, it can act as an attacker-controlled relay.